



Bessemer Tech Catalyst

Problem Statement

Prepared by: Aivar Innovations

September 2026



About Aivar Innovations

Aivar Innovations is an AI-native services and software company and AWS Preferred Partner, backed by Bessemer Venture Partners and Sorin Investments. Aivar operates three accelerator platforms — Convogent (voice and agent AI automation), Velogent (governed agentic process automation for regulated industries), and Kubogent (Kubernetes-native AIOps) — serving enterprises across fintech, healthcare, and technology verticals.

Problem Statement — AI/ML Track

Theme	AI / Machine Learning
Problem Statement	Autonomous Test Orchestration Agent
Focus Areas	Agentic AI · Developer Productivity · Software Quality Engineering

1. Background

Software testing is a discipline that consumes significant engineering time yet consistently delivers incomplete coverage. The core problem is not execution — it is decision-making: figuring out what to test, evaluating whether the right things were tested, and knowing when a failure reflects a real defect versus a broken script.

AI-assisted testing tools can now generate test plans and executable test files from a live application, and repair failing tests automatically. What they do not do is orchestrate these capabilities end to end — deciding when to plan, when to generate, when to heal, and when to escalate — without a human directing each step. Engineering teams that adopt these tools still carry the coordination burden themselves. That gap is the problem this challenge asks participants to solve.

2. Challenge

Build an autonomous test orchestration agent that takes a web application URL as input and drives the full testing lifecycle — planning, test generation, execution, and repair — without human intervention between stages.

The agent should coordinate a pipeline of three specialised sub-agents: a Planner that explores the application and produces a structured test plan, a Generator that converts the plan into executable test code with live selector validation, and a Healer that replays failing tests and repairs broken locators or flows. The meta-agent must coordinate this pipeline intelligently — evaluating coverage quality between stages, deciding when to re-plan or escalate, and synthesising all outputs into a final test quality report. Success is a system where a developer provides a URL and receives a working, meaningful test suite with no manual scripting in between.



3. Requirements

Must Have

- Accept a web application URL as the sole required input and begin the testing pipeline autonomously
- Invoke a Planner sub-agent to explore the application and produce a human-readable test plan covering meaningful user flows — not just happy paths
- Evaluate the generated plan for coverage gaps before passing it to the Generator — identifying missing flows, edge cases, and error states
- Invoke a Generator sub-agent to produce executable test files from the plan, with live selector and assertion validation
- Run the test suite and invoke a Healer sub-agent on failures, distinguishing between a broken test script and a genuine application defect
- Produce a final test quality report: scenarios covered, pass/fail outcomes, healer actions taken, coverage gaps remaining, and untested flow risk

Good to Have

- Accept an optional product requirements document to inform the Planner's scope and guide coverage decisions
- Natural language input — allow a developer to describe intent (e.g. 'focus on checkout and authentication flows') and reflect that in the plan
- Parallel test execution across multiple user flows to reduce total pipeline duration

Bonus

- PRD-to-test-plan gap analysis — after generation, compare the test plan against stated product requirements and surface what is not covered
- Defect classification — distinguish confidently between a test that is broken due to a script issue and a test that is failing due to a genuine application bug

Out of Scope

- Production deployment or hosting at scale
- CI/CD pipeline integration
- Cross-browser matrix testing
- Complete test coverage of a production application
- Manually written test scripts — all test behaviour must be produced by the agent pipeline

4. Resources Provided

The organiser may provide one or two application URLs with login credentials at the time of the event as official test targets. These will be shared on the day and should be treated as the primary validation surface for the final demo.



Teams are strongly advised not to wait for these. Bring your own test target — a self-hosted open-source app, a sample application your team has already built, or any publicly accessible demo application. The agent should work against any web application; arriving with a familiar target lets you focus on building the orchestration layer, not debugging access issues.

LLM API keys must be arranged by each team in advance. The organiser will not provide API access.

5. Evaluation Criteria

- Functionality and completeness — does the full pipeline run end to end without manual intervention? (30%)
- Innovation and originality — how intelligently does the orchestrator handle coverage gaps, ambiguity, and failure classification? (20%)
- Technical implementation and code quality — robustness of the agentic loop, quality of generated tests, and depth of the healer (20%)
- User experience and demo clarity — how clearly does the team present the agent's decisions and output? (15%)
- Business impact and feasibility — how meaningfully does the solution reduce manual QA effort for a real engineering team? (10%)
- Presentation — clarity of the live demo and ability to explain trade-offs and architecture decisions (5%)

6. Submission Requirements

- Working prototype — the orchestration agent running live on a target application
- Source code repository (GitHub or GitLab) with clear setup instructions
- README documenting the architecture, agent pipeline design, and how to run the solution
- Architecture diagram showing the orchestration flow between sub-agents
- Demo video (2 to 5 minutes) walking through the pipeline end to end
- Presentation deck covering the problem, approach, trade-offs, and business impact